

Detailed 10-Day | 40-Hour Curriculum

Advanced AI-Augmented Java Engineering w/GH Copilot

Day 1 – Foundations of AI, LLMs, Agents & MCP

Conceptual clarity for experienced engineers

What we cover

We begin by establishing a **clear mental model of modern AI**, tracing the evolution from:

- Classical AI → Machine Learning → Deep Learning
- Generative AI and why it is fundamentally different
- Large Language Models (LLMs): what they are, what they are not
- AI Agents: reasoning, tools, memory, and autonomy
- **Model Context Protocol (MCP)**: why it exists, what problem it solves, and how it changes system integration

This day intentionally **does not jump into Java code immediately**. Instead, we focus on **conceptual correctness**, terminology alignment, and architectural thinking so that participants clearly understand:

- Where GitHub Copilot fits in the AI landscape
- Where RAG, Agents, and MCP fit relative to each other
- What is hype vs what is production-relevant

How we deliver

- Whiteboard-style explanations grounded in engineering reality
- Concrete analogies from backend system design
- Mapping AI concepts to familiar backend patterns (services, adapters, gateways)
- Live demonstrations of Copilot and agent behaviour without deep coding yet

Outcome

Participants leave Day 1 with **shared vocabulary, clarity, and architectural intuition**, setting a strong foundation for all hands-on work that follows.

Day 2 – Building a Simple RAG System (Text, PDF, CSV)

Hands-on introduction to Retrieval-Augmented Generation

What we cover

This day is **purely practical**. Participants build RAG systems incrementally:

- Start with retrieval from **local text files**
- Extend to **PDF documents**
- Then structured data such as **CSV files**
- Learn how data is chunked, embedded, stored, and retrieved
- Understand the difference between naïve retrieval and semantic retrieval

We introduce:

- Embeddings and vector similarity (conceptually, not mathematically)
- Prompt templates for RAG
- Simple Java-friendly RAG workflows

How we deliver

- Step-by-step live builds
- Each data source added incrementally
- Continuous use of Copilot to scaffold ingestion, parsing, and retrieval logic
- Participants actively modify and extend the same RAG pipeline throughout the session

Outcome

Participants can **build a working RAG system** that answers questions from local enterprise documents, and clearly understand *how* and *why* it works.

Day 3 – Advanced RAG: Images, Web & MCP-Based Retrieval

From documents to real-world enterprise knowledge

What we cover

We extend RAG beyond basic documents:

- Retrieving information from **images** (OCR + vision models)
- Retrieving content from **web pages and APIs**
- Introducing **MCP-based tools as retrieval sources**
- Comparing:
 - Static RAG (documents)
 - Dynamic RAG (live tools, APIs, MCP)

We also discuss:

- When RAG breaks down
- How to reduce hallucinations
- How MCP changes the RAG architecture fundamentally

How we deliver

- Live demos of multimodal retrieval
- Designing RAG flows that mix:
 - Local data
 - External services
 - MCP tools
- Architecture discussions focused on Java backend integration

Outcome

Participants understand **enterprise-grade RAG**, including multimodal and tool-based retrieval — not just document search.

Day 4 – AI-Assisted Java Development with Copilot

From requirements to clean Java code

What we cover

We now move fully into **Java development workflows**:

- Translating requirements into Java domain models
- Interface-first and contract-driven design using Copilot
- Generating service layers, DTOs, repositories
- Understanding where Copilot accelerates design — and where human judgment is essential

MCP is referenced as a **future extension point**, not yet the focus.

How we deliver

- Live coding with Copilot as a pair programmer
- Iterative refinement instead of one-shot code generation
- Participants actively critique AI-generated code

Outcome

Participants use Copilot as a **design accelerator**, not a code generator.

Day 5 – Testing, Refactoring & Code Quality with AI

Making AI-generated code production-ready

What we cover

- Unit test generation using Copilot (JUnit)
- Identifying missing edge cases
- Refactoring for:
 - Readability
 - Performance
 - Maintainability
- Applying design patterns with AI assistance
- Secure coding considerations

How we deliver

- Refactor real Java components
- Compare “AI-first draft” vs “production-ready code”
- Continuous discussion on trust boundaries with AI

Outcome

Participants confidently **review, refactor, and harden AI-assisted Java code.**

Day 6 – Integrating LLMs into Java Applications

AI as part of the system, not the IDE

What we cover

- Calling LLM APIs from Java services
- Prompt lifecycle management
- Stateless vs stateful AI services
- Introduction to **LangChain4j** concepts
- Early use of MCP as a tool integration layer

How we deliver

- Design-focused walkthroughs
- Code scaffolding using Copilot
- Emphasis on clean separation of concerns

Outcome

Participants can **embed AI capabilities into Java applications responsibly**.

Day 7 – Enterprise Architecture, Observability & MCP

AI systems that can be monitored and trusted

What we cover

- AI-first architecture decisions
- Rules + LLM hybrid systems
- Observability:
 - Logging AI inputs/outputs
 - Tracing decisions
- Deeper dive into **MCP concepts**
- How MCP enables safe, structured tool usage

How we deliver

- Architecture diagrams
- Discussion of failure modes
- Design exercises

Outcome

Participants think about AI systems as **operational, observable software**, not black boxes.

Day 8 – DevOps, Documentation & Continuous AI

AI across the delivery lifecycle

What we cover

- Using AI to:
 - Generate and maintain JavaDocs
 - Update README and architecture docs
- AI in CI/CD:
 - Test selection
 - Failure analysis
- Continuous improvement loops
- MCP as an automation bridge

How we deliver

- Practical examples
- Workflow mapping
- Emphasis on maintainability

Outcome

Participants see how AI improves **engineering velocity beyond coding**.

Day 9 – AI Agents & Custom MCP Tools

From assistants to autonomous helpers

What we cover

- What makes an AI agent an “agent”
- Internal agents for:
 - Code review
 - Dependency analysis
 - Documentation checks
- Building **custom MCP tools**
- Defining tool contracts and context providers

How we deliver

- Agent architecture walkthroughs
- MCP tool design exercises
- Java integration strategies

Outcome

Participants understand how to **extend AI safely using MCP**, not ad-hoc scripts.

Day 10 – Capstone, Best Practices & Roadmap

Bringing it all together

What we cover

- End-to-end system thinking:
 - Java app
 - RAG
 - Agents
 - MCP
- Best practices
- Common pitfalls
- AI adoption roadmap tailored to Agilisys

How we deliver

- Capstone discussion
- Architecture reviews
- Q&A based on real Agilisys contexts

Outcome

Participants leave with **clarity, confidence, and a forward path** for AI-augmented Java engineering.

Overall Program Outcome

By the end of this program, Agilisys engineers will:

- Clearly understand **AI, LLMs, RAG, Agents, and MCP**
- Build **practical RAG systems**, not demos
- Use Copilot with intent and judgment
- Design **enterprise-grade AI-enabled Java systems**
- Achieve **sustained productivity gains**, not short-term speedups